

ADR 集合

团队名称：哪一队

日期：2026-04-28

文档版本：V1.0

目录

- [ADR-001：采用分层单体架构而非微服务架构](#)
- [ADR-002：采用 MyBatis-Plus 作为 ORM 框架](#)
- [ADR-003：采用 Redis 轻量级缓存策略，不引入独立消息队列](#)

ADR-001：采用分层单体架构而非微服务架构

状态

已接受

背景

在 Phase 2 架构设计阶段，团队需要为"Campus Helper"校园互助服务平台选择合适的架构风格。平台核心功能包括快递代取、二手交易、组队匹配、支付结算、信用评价等，目标用户为单校大学生，初期并发用户 ≤ 3000 。

关键约束条件：

- 团队规模：4人大学生团队
- 开发周期：10周（含编码、测试、部署）
- 技术储备：仅熟悉 Java 语言，需大幅依靠 AI 辅助学习
- 运维能力：无 DevOps 经验，无服务器运维经历

决策

我们决定采用**前后端分离 + 分层单体架构**（Layered Monolith），拒绝微服务架构。

理由

考虑因素	分层单体架构	微服务架构
开发复杂度	低，模块间函数调用	极高，需处理分布式事务、服务治理
团队学习成本	平缓，Spring Boot 单体开发	陡峭，需学习 K8s、注册中心、熔断限流等
10周可行性	高，1-2周搭框架，剩余时间做业务	极低，基建需2-3周，业务时间被严重挤压
运维成本	低，单台服务器 Docker 部署	高，需多台服务器、监控、日志收集
数据一致性	极易保障，单机 ACID 事务	复杂，需引入 Saga/TCC 分布式事务
扩展性	够用，单体多实例可支撑3000并发	优秀，但当前场景过度设计

核心权衡：**开发效率与可扩展性的权衡**。微服务的扩展性优势在单校3000并发场景下无法体现，而其带来的复杂度将直接导致项目无法按期交付。

后果

- **正面影响：**
 - 团队可专注于业务功能实现，而非基础设施搭建
 - 调试简单，本地一键运行，适合快速迭代
 - 部署成本低，单台阿里云 ECS 学生机即可运行
 - 数据一致性天然保障，无需处理分布式事务难题
- **负面影响：**
 - 代码集中，长期迭代可能出现"大泥球"风险
 - 无法针对单一模块独立扩容（如支付模块）
 - 技术栈统一，无法按服务选择最优语言/框架
- **需要关注的风险：**
 - 随着功能增加，需严格控制模块边界，避免循环依赖
 - 数据库单点风险，后期需考虑读写分离
 - 若未来扩展至多校运营，需预留服务拆分扩展点

AI 辅助记录

AI 初稿内容摘要

成员A（直接提问）的 AI 回答推荐：**前后端分离 + 微服务架构**，技术栈为 Go + Gin 框架，服务拆分为用户、任务、订单、支付、消息等独立微服务，使用 Nacos 做服务发现，PostgreSQL + PostGIS 处理地理位置。

AI 理由：微服务"兼顾高性能与可扩展性"，是"当前业界流行方案"。

人工修订内容

1. **推翻架构选择：**从 AI 推荐的微服务改为分层单体架构
2. **修正技术栈：**从 Go 语言改回团队P0阶段自主选择的 Java + Spring Boot
3. **删减过度组件：**移除 Nacos、PostGIS、RabbitMQ 延时消息等

4. 补充约束分析：增加"10周可行性"、"团队学习成本"等 AI 未考虑的维度

修订理由

AI 初稿存在以下问题，导致必须人工修订：

1. **忽略约束条件**：AI 未询问团队规模、时间限制、技术储备，默认按"理想团队"假设输出
2. **技术栈不匹配**：推荐 Go 语言与团队 Java 背景冲突，10 周无法完成语言学习
3. **过度设计**：Nacos、PostGIS 等组件对 3000 并发场景完全多余，增加不必要的学习成本
4. **遗漏关键质量属性**：AI 强调"可扩展性"但忽略"开发效率"和"时间约束"，而时间约束是学生项目最关键的成败因素

核心判断：AI 无法替团队承担"项目能否按期交付"的责任，这个决策必须基于团队实际能力人工做出。

ADR-002：采用 MyBatis-Plus 作为 ORM 框架

状态

已接受

背景

在确定分层单体架构后，需要选择后端数据访问层的 ORM 框架。候选方案包括：

- MyBatis-Plus：基于 MyBatis 的增强工具，简化 CRUD
- Spring Data JPA：Spring 官方推荐的 ORM 规范实现
- 纯 MyBatis：手写 SQL，灵活性最高但样板代码多

团队约束：

- 有时需依靠AI辅助生成代码框架和调整代码
- 团队成员对 SQL 理解程度不一
- 10 周内需完成核心业务功能，无法花太多时间学习框架

决策

我们决定采用 **MyBatis-Plus 3.5+** 作为 ORM 框架，不选用 JPA。

理由

考虑因素	MyBatis-Plus	Spring Data JPA
学习曲线	平缓，SQL 直观可见	较陡，需理解 ORM 映射、HQL、Specification
AI 辅助友好度	高，AI 生成 SQL 易于理解和修改	中，AI 生成 JPQL 准确性较低，易出现性能问题
复杂查询控制	优秀，原生 SQL + 条件构造器	一般，复杂查询需手写 @Query 或原生 SQL
代码生成	内置 Generator，一键生成 Entity/Mapper/Service	需配合 Lombok 手动编写或额外工具
团队匹配度	高，与 Spring Boot 集成成熟	中，团队无 JPA 使用经验

核心权衡：**开发效率与抽象程度的权衡**。JPA 的"约定优于配置"在企业级开发中有优势，但对需要 AI 辅助、SQL 基础薄弱的学生团队，MyBatis-Plus 的"所见即所得"更可控。

后果

- **正面影响：**
 - 代码生成器大幅减少样板代码编写时间
 - SQL 直观，便于 AI 辅助生成和人工审查
 - 复杂查询（如订单状态统计、信用分计算）可控性强
 - 社区活跃，中文文档丰富，遇到问题易解决
- **负面影响：**
 - 需手动管理表结构变更（无 JPA 的自动 DDL）
 - 数据库迁移需引入 Flyway/Liquibase 等额外工具
 - 代码量略多于 JPA（需写 Mapper XML 或注解）
- **需要关注的风险：**
 - 团队成员可能写出 N+1 查询，需注意关联查询优化
 - 代码生成器生成的代码需审查，避免冗余字段暴露
 - 需建立统一的 SQL 编写规范，避免风格混乱

AI 辅助记录

AI 初稿内容摘要

成员D（扮演架构师）的 AI 回答推荐：**MyBatis-Plus**，理由为"减少样板代码，复杂 SQL 可控"。

但 AI 同时提到："如果团队更熟悉 JPA，也可以选用 Spring Data JPA，两者都是主流方案"。

人工修订内容

1. **明确排除 JPA：**AI 的"两者都可以"建议过于模糊，人工明确选定 MyBatis-Plus
2. **增加 AI 辅助维度：**补充"AI 生成 SQL 的可控性"这一团队关键约束

3. 补充负面影响：AI 只提优点，人工补充了 DDL 管理、N+1 查询等风险

修订理由

AI 初稿的问题：

1. **回避决策**：AI 倾向于给出"两者都可以"的安全回答，避免承担选择责任。但项目必须做出明确选择，不能同时维护两套方案。
2. **遗漏团队约束**：AI 未考虑"团队需大幅依靠 AI 辅助"这一核心约束。JPA 的抽象层会增加 AI 理解难度（如关联映射、懒加载），而 MyBatis-Plus 的 SQL 直观性更利于 AI 辅助。
3. **忽略质量属性权衡**：AI 强调"减少样板代码"但忽略"可维护性"和"可审查性"。学生项目需要代码可被团队成员和助教理解，MyBatis-Plus 的 SQL 可见性更符合这一需求。

核心判断：AI 可以列出技术优缺点，但无法评估"哪种技术更适合需要 AI 辅助的初学者团队"，这个判断必须基于团队实际能力人工做出。

ADR-003：采用 Redis 轻量级缓存策略，不引入独立消息队列

状态

已接受

背景

在架构设计过程中，需要确定缓存策略和异步处理方案。存在以下需求：

- 用户会话存储（JWT Token 黑名单）
- 短信验证码缓存（5分钟过期）
- 热点数据缓存（任务列表、用户信用分）
- 订单超时自动取消（发布中2小时未接单、待确认2天未确认）

候选方案：

- **方案A**：Redis 全面使用 + 引入 RabbitMQ/Kafka 处理延时消息（订单超时）
- **方案B**：Redis 轻量级使用（缓存/会话/分布式锁）+ Spring 定时任务处理订单超时
- **方案C**：不用 Redis，纯数据库 + 内存缓存（Caffeine）

团队约束：

- 10 周时间紧张，需减少组件学习成本
- 无消息队列使用经验
- 单校场景，数据量和并发量可控

决策

我们决定采用**方案B：Redis 轻量级使用**，具体策略：

用途	实现方式	不引入独立 MQ 的替代方案
会话存储	Redis Hash	—
验证码	Redis String（5分钟 TTL）	—
热点数据	Redis String	—
分布式锁	Redisson	—
订单超时取消	Redis Key 过期监听 + Spring 定时任务补偿	不用 RabbitMQ 延时队列

理由

考虑因素	方案B（选定）	方案A（RabbitMQ）	方案C（不用Redis）
学习成本	低，Redis 命令简单	高，需学习 MQ 概念、交换机、队列绑定	极低
部署复杂度	低，Docker 单容器	高，需额外部署 MQ，配置持久化	极低
订单超时精度	分钟级（定时任务轮询）	秒级（延时队列）	分钟级
数据一致性风险	低	中（MQ 消息丢失、重复消费）	低
10周可行性	高	中（增加2-3天学习+调试时间）	高

核心权衡：**精度与复杂度的权衡**。订单超时取消不需要秒级精度（2天的超时，分钟级轮询完全可接受），为此引入 RabbitMQ 的学习和运维成本不划算。

后果

- **正面影响：**
 - 减少一个中间件（RabbitMQ）的学习和部署成本
 - Redis 一器多用（缓存+会话+锁+延时），降低系统复杂度
 - Spring 定时任务内置支持，无需额外依赖
 - 调试简单，超时逻辑在代码中直观可见
- **负面影响：**
 - 订单超时精度为分钟级（定时任务轮询间隔），非秒级
 - 定时任务在单体多实例部署时需考虑重复执行（可通过 Redis 分布式锁解决）
 - 未来订单量激增时，定时任务可能成为性能瓶颈
- **需要关注的风险：**

- 定时任务轮询间隔设置需合理（建议1分钟），避免频繁查询数据库
- 需记录定时任务执行日志，便于排查超时未触发问题
- 若未来需要秒级精度（如秒杀场景），需迁移至延时队列

AI 辅助记录

AI 初稿内容摘要

成员D（扮演架构师）的 AI 回答推荐：**RabbitMQ / Redis Stream** 作为消息队列，用于"异步解耦（通知、积分计算）"，理由是"团队学习成本低"。

成员B（提供约束）的 AI 回答则明确：**放弃 IM 系统，用订单留言代替**，但未明确提及消息队列。

人工修订内容

1. **排除 RabbitMQ**：AI 推荐的 RabbitMQ 被人工排除，改用 Redis + 定时任务
2. **重新定义 Redis 用途**：从 AI 的"消息队列（Redis Stream）"改为"轻量级缓存+会话+锁"
3. **补充精度分析**：增加"分钟级精度足够"的业务判断
4. **增加风险预案**：补充未来迁移至延时队列的扩展点

修订理由

AI 初稿的问题：

1. **过度推荐组件**：AI 倾向于"推荐更多组件以展示全面性"。成员D的 RabbitMQ 推荐对 10 周项目明显过度，成员B 虽未推荐 MQ 但也未明确排除。
2. **忽略精度需求**：AI 未分析订单超时取消的实际精度需求。2天的超时容忍分钟级误差，无需引入 MQ 的秒级精度。
3. **"学习成本低"的判断错误**：AI 声称 RabbitMQ"团队学习成本低"，但实际需要理解交换机类型、队列绑定、消息确认、死信队列等概念，对初学者并不低。
4. **遗漏运维成本**：AI 只考虑开发成本，忽略 MQ 的部署、监控、消息堆积处理等运维成本，而学生项目无运维人力。

核心判断：AI 无法评估"某个组件对特定团队是否值得引入"，因为它不理解"10周"的真正含义——不是"10周内学会"，而是"10周内交付可演示的系统"。这个减法决策必须人工做出。

文档编制：LXY

审核状态：已经由人工审查